

Local AI Orchestrator

Technical Reference

Every file. Every service. Every table. Every state location.
Use as a reference; do not read end-to-end. Jump to whatever you want to understand.

Generated: 2026-05-18

Table of contents

1. How to read this document
2. Service: Docker Compose
3. Service: PostgreSQL
4. Service: Temporal
5. Service: Ollama
6. Service: LiteLLM
7. Service: Open WebUI
8. Service: Streamlit control panel
9. Service: Temporal worker (Python long-running)
10. Repository: root-level files
11. Repository: `assistant_core/__init__.py` and top-level modules
12. Repository: `assistant_core/clarification/`
13. Repository: `assistant_core/execution/`
14. Repository: `assistant_core/llm/`
15. Repository: `assistant_core/memory/`
16. Repository: `assistant_core/temporal_app/`
17. Repository: `app/control_panel.py`
18. Repository: `scripts/` (every shell script)
19. Repository: `config/` (every YAML)
20. Repository: `db/migrations/`
21. Repository: `tests/` (every test file)
22. Repository: `docs/`
23. External state: `~/LocalAI/`
24. External state: `~/Obsidian/LocalAI-ChiefOfStaff/`
25. State files: `day_unlock.flag` and others
26. Environment variables in depth
27. The complete port map
28. Logging: where logs go, what they contain

29. Inspection cheatsheet: SQL + docker + ollama

1. How to read this document

This is a reference, not a tutorial. The System Guide explains the *shape* of the system and the Onboarding shows you *how to use* it; this document tells you *what every individual piece does* and why it is there. It is structured by entity, not by use case. Jump to whatever section answers the question you have right now.

Three levels of detail in three documents

Document	Audience question it answers	Reading mode
System Guide	"What is this system? How does it think?"	Read once, cover to cover.
Onboarding	"How do I use this thing today?"	Skim once, then return for recipes.
Technical Reference (this doc)	"What exactly does THIS file/service/table do?"	Reference. Search. Never read end-to-end.

Conventions used below

- **Path:** the file path relative to project root, or absolute for things outside the repo.
- **What it does:** literal description, present tense, what the entity is responsible for.
- **Why it exists:** design rationale; what would break or be missing without it.
- **How to inspect:** a command, query, or path to read what's currently true.

2. Service: Docker Compose

What Docker is doing for us

Docker is the container runtime; **Docker Compose** is the multi-service orchestrator. In this project we use Compose to start, stop, and configure four stateful services as containers running side-by-side on your Mac: **Postgres**, **Temporal**, **Temporal UI**, and **Open WebUI**. Each container is an isolated process tree with its own filesystem, but they share a private Docker network so they can reach each other by name.

Why containers (rather than brew install)

Postgres and Temporal have non-trivial initialization (schema setup, dynamic config, startup ordering). Containers package the init logic, environment, and the right version. Running them via Compose means: same versions across machines, no host pollution, one command up, one command down, named volumes to persist data even when the container is destroyed.

The compose file in detail

docker-compose.yml at the repo root defines four services. Highlights:

Service	Image	Host port (127.0.0.1)	Container port	Purpose
postgres	postgres:16	5432	5432	Operational state DB
temporal	temporalio/auto-setup:1.26	7233	7233	Workflow engine gRPC
temporal-ui	temporalio/ui:2.31.2	8233	8080	Web UI for Temporal
open-webui	ghcr.io/open-webui/open-webui:main	8000	8080	Chat UI against Ollama

Networks

Compose auto-creates a private bridge network named local-ai-orchestrator_default. Containers reach each other by service name: Temporal connects to postgres:5432 internally, Temporal UI to temporal:7233, Open WebUI to host.docker.internal:11434 (the host's Ollama). Host ports are all bound to 127.0.0.1 only — nothing exposed to the LAN.

Volumes

Volume	Container path	What it holds
postgres_data	/var/lib/postgresql/data	Your work packets, runs, audit data. Survives container recreation.
temporal_data	/etc/temporal	Temporal config (mostly bundled in image).
open_webui_data	/app/backend/data	Open WebUI's chat history and user settings.
./db/migrations (bind mount)	/docker-entrypoint-initdb.d	Read-only. SQL files run by Postgres on first boot.

Important behaviors

- **First boot:** Postgres sees db/migrations/001_init.sql mounted in its init dir, runs it, creates the seven tables.
- **Subsequent boots:** Postgres ignores init scripts (data dir already initialized). Schema changes require new migration files (or DROP + re-mount).
- **Healthchecks:** postgres has a healthcheck via pg_isready. temporal depends_on postgres with condition: service_healthy so it waits for the DB.
- **The Temporal dynamic-config bug:** the compose was originally pointing at config/dynamicconfig/development-sql.yaml (doesn't exist in image 1.26). Fixed to docker.yaml in commit a2d6485.
- **stop_services.sh stops these four containers ONLY.** Streamlit, Ollama, and host Python processes are not affected.

How to inspect

```
docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
docker logs --tail=50 localai-postgres
docker logs --tail=50 localai-temporal
docker compose config      # print the resolved compose file
docker volume ls           # list named volumes
docker network inspect local-ai-orchestrator_default
```

3. Service: PostgreSQL

What Postgres is doing for us

PostgreSQL is the orchestrator's **operational state store**. It holds every work packet you create, every clarification question generated, every overnight run, every model call, every evaluation, every artifact pointer, and every memory candidate. It does *not* hold your input notes (those stay in ~/LocalAI/inbox/) or the long-term knowledge layer (that's Obsidian). It holds the orchestrator's view of its own work.

Why Postgres (rather than SQLite or files)

Three reasons: durable relational integrity (work packets, questions, runs, and model calls all reference each other and we want foreign keys enforced); JSON columns (policies are stored as JSONB so we can evolve schema without migrations); and Temporal needs Postgres anyway for its own state, so running one DB serves both.

Connection details

Property	Value
Connection string	postgresql://localai:localai@localhost:5432/localai
User	localai
Password	localai (local-only; don't expose this DB)
Database name	localai
Postgres version	16 (postgres:16 image)
Container name	localai-postgres

Schema overview (7 tables)

Table	Rows =	Relations
work_packets	one per request you create	parent to everything else
clarification_questions	one per question generated	FK -> work_packets
execution_runs	one per overnight invocation	FK -> work_packets
model_calls	one per model API call	FK -> work_packets, execution_runs
evaluations	one per evaluator pass	FK -> work_packets, execution_runs
artifacts	one per generated file	FK -> work_packets, execution_runs
memory_candidates	one per candidate from outputs	FK -> artifacts

Table: work_packets — the central record

```
CREATE TABLE work_packets (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  title TEXT NOT NULL,           -- your title at creation  
  objective TEXT,                -- the description / objective  
  status TEXT NOT NULL,          -- DRAFT, UNDERDEFINED,  
                                -- NEEDS_CLARIFICATION,  
                                -- READY_FOR_OVERNIGHT,  
                                -- READY_HIGH_CONFIDENCE,  
                                -- PAUSED, RESUME_REQUESTED,  
                                -- RUNNING, COMPLETED, FAILED  
  readiness_score NUMERIC,       -- 0.0 - 1.0  
  high_stakes BOOLEAN DEFAULT FALSE,  
  cloud_policy JSONB,            -- e.g. {allowed: false, explicit: true}  
  source_policy JSONB,          -- e.g. {scope: '~/LocalAI/inbox/notes'}  
  output_policy JSONB,          -- e.g. {format: 'markdown brief'}  
  created_at TIMESTAMPTZ DEFAULT now(),  
  updated_at TIMESTAMPTZ DEFAULT now(),  
  raw_user_request TEXT,         -- the description you typed  
  structured_yaml TEXT           -- yaml-serialized full pydantic model  
);
```

Table: clarification_questions

```
CREATE TABLE clarification_questions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  work_packet_id UUID REFERENCES work_packets(id),  
  round_number INTEGER,          -- 1 = first question round; can grow if rescored  
  category TEXT,                 -- 'objective', 'outputs', 'sources', 'privacy/cloud',  
                                -- 'quality', 'audience', 'assumptions',  
                                -- 'escalation', 'stop conditions'  
  question TEXT,                 -- the strong-question text  
  priority INTEGER,              -- 50 - 100  
  blocking BOOLEAN,              -- true for the 4 hard dimensions  
  answered BOOLEAN DEFAULT FALSE,  
  answer TEXT,                   -- the raw markdown answer block  
  created_at TIMESTAMPTZ DEFAULT now(),  
  answered_at TIMESTAMPTZ  
);
```

Table: execution_runs

```
CREATE TABLE execution_runs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  work_packet_id UUID REFERENCES work_packets(id),  
  temporal_workflow_id TEXT,     -- the workflow id submitted to Temporal  
  status TEXT,                   -- RUNNING, COMPLETED, FAILED, CANCELLED  
  mode TEXT,                     -- DAY_MODE, NIGHT_MODE, MANUAL_RESUME  
  started_at TIMESTAMPTZ,  
  completed_at TIMESTAMPTZ,  
  error_summary TEXT             -- last error message, if any  
);
```


Table: model_calls (audit)

```
CREATE TABLE model_calls (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  execution_run_id UUID REFERENCES execution_runs(id),  
  work_packet_id UUID REFERENCES work_packets(id),  
  file_path TEXT,           -- the source file this call is about  
  task_type TEXT,           -- 'general', 'coding', 'reasoning_check', ...  
  model_group TEXT,         -- 'local-main', 'local-coder', ...  
  actual_model TEXT,        -- e.g. 'qwen3:30b-a3b'  
  local_or_cloud TEXT,      -- 'local' or 'cloud'  
  prompt_chars INTEGER,  
  response_chars INTEGER,  
  success BOOLEAN,  
  error TEXT,  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

Table: evaluations

```
CREATE TABLE evaluations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  execution_run_id UUID REFERENCES execution_runs(id),  
  work_packet_id UUID REFERENCES work_packets(id),  
  file_path TEXT,  
  quality_score NUMERIC,    -- 0.0 - 1.0  
  grounding_score NUMERIC,  
  completeness_score NUMERIC,  
  actionability_score NUMERIC,  
  contradiction_score NUMERIC,  
  hallucination_risk TEXT,  -- 'low', 'medium', 'high'  
  needs_retry BOOLEAN,  
  recommended_next_step TEXT, -- 'pass', 'retry_local_secondary',  
                               -- 'reasoning_check', 'cloud_review',  
                               -- 'human_review'  
  reason TEXT,  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

Table: artifacts

```
CREATE TABLE artifacts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  execution_run_id UUID REFERENCES execution_runs(id),  
  work_packet_id UUID REFERENCES work_packets(id),  
  artifact_type TEXT,      -- 'morning_brief', 'priorities', etc.  
  file_path TEXT,          -- absolute path under ~/LocalAI/output/  
  obsidian_path TEXT,      -- absolute path inside vault  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

Table: memory_candidates

```
CREATE TABLE memory_candidates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  source_artifact_id UUID REFERENCES artifacts(id),
  candidate_text TEXT,
  destination_suggestion TEXT,      -- where in the vault it should land
  confidence TEXT,                  -- 'low', 'medium', 'high'
  memory_type TEXT,                 -- 'confirmed', 'derived', 'inferred'
  approved BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

Indexes

Index	Speeds up
idx_work_packets_status	filtering by status (READY, RUNNING, etc.)
idx_questions_work_packet	loading all questions for one packet
idx_execution_runs_work_packet	loading all runs for one packet
idx_artifacts_work_packet	listing artifacts of one packet

How to inspect

```
docker exec -it localai-postgres psql -U localai -d localai

-- inside psql:
\dt          -- list tables
\d work_packets -- describe a table
SELECT id, title, status, readiness_score
  FROM work_packets ORDER BY created_at DESC LIMIT 10;
SELECT count(*) FROM clarification_questions WHERE answered = false;
```

4. Service: Temporal

What Temporal is doing for us

Temporal is a **durable workflow engine**. Where a regular Python script would lose its place if the process crashed mid-execution, a Temporal workflow records every state change to its database and can resume exactly where it left off. In this project Temporal is the runtime for overnight execution: it accepts workflow submissions, runs them across long-running Python *activities*, retries on failure, and provides a UI to observe everything that ever ran.

Why Temporal (not cron + bash)

Overnight runs may take hours, talk to flaky local services (Ollama can stall, models can fail), and need clean restart semantics if your Mac sleeps mid-job. Cron + bash gives you none of that. Temporal gives you: per-activity retries, timeouts, idempotency, full audit history, pause and resume, and a web UI showing every workflow execution.

Two pieces

Piece	Lives in	Started by
Temporal server (gRPC + UI)	Docker container (auto-setup image)	bash scripts/start_services.sh
Temporal worker (Python)	Host Python process	bash scripts/start_temporal_worker.sh

The **server** stores workflow state in Postgres (separate tables, different namespace from our own seven tables). The **worker** is a long-running Python process that polls the server for work to do; the work items are calls to activity functions like `create_work_packet_activity`.

Workflows (defined in code)

Workflow	Where	What it does
ClarificationWorkflow	assistant_core/temporal_app/workflows.py	One activity: <code>create_work_packet_activity</code> . Uses <code>generate_questions_activity</code> to get questions for a description.
OvernightExecutionWorkflow	same file	Three activities: <code>update_status</code> to RUNNING, <code>score_readiness</code> , and <code>generate_questions</code> .
ManualResumeWorkflow	same file	Reserved for v2 user-driven re-runs of paused workflows.

Activities (the Python functions Temporal calls)

Activity name	What it does
<code>create_work_packet_activity</code>	Builds a packet via deterministic clarification, writes to Postgres.
<code>generate_questions_activity</code>	Generates questions for a description (returns JSON).
<code>score_readiness_activity</code>	Scores readiness, returns the structure.

Activity name	What it does
run_langgraph_execution_activity	(scaffolded) Runs the 18-step execution graph for a ready packet.
write_outputs_activity	(scaffolded) Writes generated artifacts to ~/LocalAI/output/.
update_status_activity	(scaffolded) Updates work_packet status in Postgres.
archive_files_activity	(scaffolded) Moves processed inputs to ~/LocalAI/archive/.

Task queue

All workflows are submitted against the single task queue named local-ai-orchestrator. The worker polls this queue and dispatches activities. If no worker is running, workflows queue but never execute — you can see them in the Temporal UI marked as scheduled.

Temporal UI

Browse to <http://localhost:8233>. You'll see the default namespace, a list of workflows, and per-workflow event history with timing, retries, and inputs/outputs. Great for debugging.

How to inspect from the terminal

```
# Server health
docker logs --tail=30 localai-temporal

# Is the worker running on the host?
ps -ef | grep '[t]emporal_app.worker'

# Submit a manual clarification workflow from Python:
.venv/bin/python -c "
from assistant_core.temporal_app.client import start_clarification
print(start_clarification('Test', 'A vague description'))
"
```

5. Service: Ollama

What Ollama is doing for us

Ollama is the **local model runtime**. It downloads quantized weights to `~/.ollama/models/`, loads them into Metal VRAM on demand, and exposes an HTTP API on `:11434` for inference, listing, and load/unload control. The orchestrator never bypasses Ollama for local model calls; every local inference goes through this daemon.

Why Ollama (not raw llama.cpp)

llama.cpp is the underlying inference engine, but operating it directly means tracking weights, format conversions, quantization options, GPU settings, and writing your own HTTP server. Ollama wraps all of that, ships a registry of pre-quantized models, and gives you a simple HTTP API. It's the right level of abstraction for a single-user Mac.

Where things live

Path	Contents
<code>~/.ollama/models/blobs/</code>	Raw quantized weight blobs, content-addressable.
<code>~/.ollama/models/manifests/</code>	Manifests linking tags (e.g. qwen3:30b-a3b) to blob digests.
<code>~/.ollama/logs/server.log</code>	Daemon logs (also visible via <code>brew services log ollama</code>).
<code>~/Library/LaunchAgents/homebrew.mxcl.ollama.plist</code>	macOS launchd config when running as brew service.

Key API endpoints

Endpoint	What it does	Used by
<code>GET /api/tags</code>	Lists tags on disk	<code>ollama_admin.list_pulled_models</code>
<code>GET /api/ps</code>	Lists tags currently in VRAM	<code>ollama_admin.list_loaded_models</code>
<code>POST /api/generate</code>	Run inference (also used for load/unload)	<code>ollama_admin.load_model</code> , <code>unload_model</code> , <code>quick_prompt</code>
<code>POST /api/pull</code>	Pull a tag from registry	<code>ollama pull</code> CLI
<code>POST /api/delete</code>	Remove a tag from disk	<code>ollama rm</code> CLI

keep_alive semantics (the subtle one)

When you `POST /api/generate` you can include a `keep_alive` field telling Ollama how long to keep the model in VRAM after the call:

Value	Meaning	Used here as
"5m", "30m", "2h"	Go duration: stay loaded that long after the last call	load button (30m), quick_prompt (5m)
"0s" (string)	Unload immediately after this call	Unload button (CORRECT form)
0 (integer)	Quirky: some versions treat as "use default 5m"	Avoided — was a bug we fixed in 4480d5a
-1 (negative)	Stay loaded indefinitely	Not used here
omitted	Use server default (5m)	Default Ollama behavior

Model-tag conventions in this project

Logical name (group)	Ollama tag	Approx size on disk	VRAM when loaded
local-main	qwen3:30b-a3b	~18 GB	~30 GB
local-secondary	llama3.3:70b	~40 GB	~42 GB
local-coder	qwen3-coder:30b	~17 GB	~30 GB
local-reasoner	deepseek-r1:8b	~5 GB	~5 GB

How to inspect

ollama list	# tags on disk
ollama ps	# tags currently in VRAM
curl http://localhost:11434/api/tags	# same, JSON
curl http://localhost:11434/api/ps	# same, JSON
ollama show qwen3:30b-a3b	# detailed model metadata
brew services list grep ollama	# is it running as a service?
du -sh ~/.ollama/models	# how much disk are tags using?

6. Service: LiteLLM

What LiteLLM is doing for us

LiteLLM is a **model router**. It speaks the OpenAI chat-completion API and dispatches each call to the right backend — in our case mostly Ollama, but it can also route to Anthropic, OpenAI, or other providers. It centralizes routing rules, fallback chains, and translates a single `model_group` name into the actual backend call.

Why LiteLLM (when we already have `ollama_admin`?)

Two reasons: First, when the execution graph is fully wired, code calling `local-main` shouldn't have to know whether that's an Ollama tag or a cloud endpoint — LiteLLM hides that. Second, fallback chains (e.g., if `local-coder` fails, try `local-main`) are easier to express in LiteLLM's declarative config than in custom Python. The `ollama_admin` module is a direct path for admin operations (load/unload/test from the panel); LiteLLM is for the production execution path that calls models repeatedly.

Configuration: `config/litellm.yaml`

Gitignored; copied from `config/litellm.yaml.example` on first run. Contains the model list and the fallback rules:

```
model_list:
  - model_name: local-main
    litellm_params:
      model: ollama/qwen3:30b-a3b
      api_base: http://localhost:11434
  - model_name: local-secondary
    litellm_params:
      model: ollama/llama3.3:70b
      api_base: http://localhost:11434
  - model_name: local-coder
    litellm_params:
      model: ollama/qwen3-coder:30b
      api_base: http://localhost:11434
  - model_name: local-reasoner
    litellm_params:
      model: ollama/deepseek-r1:8b
      api_base: http://localhost:11434
  - model_name: cloud-claude-opus
    litellm_params:
      model: anthropic/claude-opus-4-7
      api_key: os.environ/ANTHROPIC_API_KEY

router_settings:
  fallbacks:
    - local-main:
      - local-secondary
    - local-coder:
      - local-main
    - local-reasoner:
```

```
- local-main

litellm_settings:
  request_timeout: 180
  set_verbose: false
```

Important design choices

- **Cloud is in the model_list but not in any fallback chain.** cloud-claude-opus is declared so it can be called if every safety gate passes, but it is intentionally not chained behind local. No blind escalation.
- **Local fallbacks are layered to avoid loops.** local-coder -> local-main is fine; we don't chain back to local-coder.
- **request_timeout: 180s** matches the longest expected single-call duration on a beefy local model.

How to inspect

```
lsof -nP -iTCP:4000 -sTCP:LISTEN    # is LiteLLM running?
curl http://localhost:4000/v1/models # what models does it advertise?
tail -f logs/litellm.log            # if you redirect when starting
```


7. Service: Open WebUI

What Open WebUI is doing for us

Open WebUI is a polished, ChatGPT-style web UI for chatting with local models. It speaks directly to Ollama via `http://host.docker.internal:11434`. It is **not part of the orchestrator's pipeline**; it lives next to it as a convenience.

Why include it

The orchestrator deliberately does not offer chat. Sometimes you want chat anyway — for spot-checks, exploration, or interactive sessions. Open WebUI is the right tool for that and co-installing it via `docker-compose` means both surfaces start with one command.

Configuration

Environment variable	Value	Meaning
OLLAMA_BASE_URL	<code>http://host.docker.internal:11434</code>	How the container reaches the host's Ollama
WEBUI_AUTH	<code>"false"</code>	No login required (single-user local box)

What is stored

Chat history, user settings, and any custom prompts you save live inside the `open_webui_data` Docker volume mounted at `/app/backend/data`. Survives container recreates. To wipe: `docker compose down -v` (also nukes your work packets — be careful).

Relationship to the orchestrator

They share Ollama and that's it. Open WebUI does not read or write the orchestrator's Postgres, does not see work packets, does not respect `DAY_MODE` / `day_unlock`. If you load a model in Open WebUI for a chat, the orchestrator's Models tab will show it as loaded too (both call the same Ollama).

8. Service: Streamlit control panel

What the panel is doing for us

The Streamlit control panel is the orchestrator's **graphical UI**. It is the only browser surface for safe model load/unload, schedule editing, and work-packet status. It is a Python process running on the host (not in Docker), serving an interactive web app on :8501.

Why Streamlit

Streamlit lets us write a UI in pure Python without managing a separate frontend. Every interaction is a server-side re-render; we can call into `assistant_core` directly with no API layer. Tabs and forms are stateless across re-renders, which fits the orchestrator's design (read state from disk or DB on every render).

Tab-by-tab

Tab	Calls into	Purpose
Dashboard	<code>storage.list_work_packets</code>	Table of all packets and their status
Create Work Packet	<code>clarification.graph.run_deterministic_clarification</code>	Draft + score a packet interactively
Clarification	(file upload only)	Preview an answers file (parse from CLI today)
Models	<code>llm.ollama_admin</code>	Load / Unload / Quick test, gate-checked
Schedule	<code>config_writer.update_assistant_config</code> + <code>scheduler.update_states</code>	Edit states, view launchd state
Execution	<code>execution.graph.describe_execution_scaffold</code>	Show the scaffold list of 18 steps
Artifacts	filesystem walk of <code>cfg.output_dir</code>	List files in <code>~/LocalAI/output/</code>

Header status strip (rendered on every page)

- **Row 1:** Current mode, Day unlock, Now (clock time + DAY/NIGHT window label), Auto-execution.
- **Row 2:** Postgres up/down, Temporal up/down, Ollama up/down, LiteLLM up/down.
- **Day-unlock banner:** red error block with flag-file contents when unlock is ACTIVE.
- **Launchd banner:** yellow warning when the nightly job is loaded in launchctl.
- **Cloud warning:** always-on green block reminding you cloud is locked behind six gates.

How to inspect

```
lsof -nP -iTCP:8501 -sTCP:LISTEN          # is Streamlit running?
tail -f /tmp/streamlit.log                 # if launched with nohup ... > /tmp/.
..
ps -ef | grep '[s]treamlit'               # PID and command line
```

9. Service: Temporal worker (long-running Python)

The Temporal worker is a Python process that polls the Temporal server's local-ai-orchestrator task queue and executes activities. Without it, workflows submitted to Temporal queue forever. It is a **separate process** from the Streamlit panel and the CLI.

Started by

```
bash scripts/start_temporal_worker.sh
# Equivalent:
.venv/bin/python -m assistant_core.temporal_app.worker
```

What it imports and registers

- Workflows: ClarificationWorkflow, OvernightExecutionWorkflow, ManualResumeWorkflow.
- Activities: create_work_packet_activity, generate_questions_activity, score_readiness_activity, run_langgraph_execution_activity, write_outputs_activity, update_status_activity, archive_files_activity.
- All registered against the queue local-ai-orchestrator on the Temporal client connected to localhost:7233.

What it doesn't do today

The activity bodies for execution (the actual model-calling, evaluator-running, file-writing work) are stubs returning placeholder dicts. Submitting an overnight workflow today works: the worker receives it, runs the stubs, marks them complete. No real outputs are produced yet. Wiring the real bodies is the next major feature chunk.

Running and observing

```
# In a dedicated terminal so its logs are visible:
bash scripts/start_temporal_worker.sh
# It prints "Temporal worker listening on task queue: local-ai-orchestrator"
# and stays running. Ctrl+C to stop.

# To run it in background:
nohup bash scripts/start_temporal_worker.sh > ~/LocalAI/logs/worker.log 2>&1 &

# Verify it's running:
ps -ef | grep '[t]emporal_app.worker'
```

10. Repository: root-level files

File	Purpose
README.md	First impression / install / ports / safety summary. Read by humans landing on the repo.
AGENTS.md	Rules for any future agent (human or AI) modifying this repo. Conservative defaults, no des
ONBOARDING.md	Practical day-to-day operator guide. Companion to the Onboarding PDF.
.env.example	Template of environment variables. Copy to .env and customize. Real .env is gitignored.
.gitignore	Lists: .env, __pycache__, config/litellm.yaml, logs/*.log, launchd/*.plist, etc.
pyproject.toml	Python project metadata + dependency list. Used by install_core.sh.
docker-compose.yml	The four-service stack: postgres, temporal, temporal-ui, open-webui.
.streamlit/config.toml	Streamlit defaults (disables usage stats gathering).
.pytest_cache/	pytest's cache (gitignored).
.venv/	The Python virtual env install_core.sh creates (gitignored).

11. Repository: assistant_core top-level modules

The Python package that holds the orchestrator's logic. Top-level modules (not subpackages) below:

Module	Responsibility
<code>__init__.py</code>	Marks assistant_core as a Python package. Empty.
<code>cli.py</code>	argparse-based CLI: ensure-folders, sample-readiness, migrate, create-work-packet, run-clarification
<code>config.py</code>	Loads config/assistant.yaml + config/memory.yaml into pydantic models (AssistantConfig, Memo
<code>config_writer.py</code>	Safely writes back to config/assistant.yaml. Enforces an allow-list (only day/night time fields edita
<code>logging_utils.py</code>	(stub) Future home for structured logging helpers.
<code>paths.py</code>	LOCALAI_SUBDIRS, OBSIDIAN_SUBDIRS, ensure_local_folders(), path-safety helpers (is_rela
<code>safety.py</code>	The five gate functions: assert_cloud_allowed, assert_no_external_write, assert_original_file_wri
<code>schemas.py</code>	All pydantic models: WorkPacket, ClarificationQuestion, ReadinessScore, EvaluationResult, Moc
<code>scheduler_status.py</code>	Service health probes (postgres_up, temporal_up, ollama_up, litellm_up). launchd inspection (la
<code>storage.py</code>	Postgres CRUD: create_work_packet, get_work_packet, update_work_packet, save_questions,

12. Repository: assistant_core/clarification/

The implemented heart of the orchestrator. Everything in this subpackage is deterministic; no model calls.

File	Responsibility
<code>__init__.py</code>	Marker file.
<code>work_packet_builder.py</code>	<code>build_initial_work_packet(title, description)</code> creates the first <code>WorkPacket</code> . <code>update_packet_from_</code>
<code>readiness.py</code>	<code>READINESS_WEIGHTS</code> dict (the 9 dimensions). <code>_dimension_scores</code> walks each dimension. <code>s</code>
<code>question_generator.py</code>	<code>QUESTION_BANK</code> : dict of category -> (category_name, question_text, priority, blocking). <code>gene</code>
<code>graph.py</code>	<code>run_deterministic_clarification(title, description)</code> ties together build + score + generate in one ca
<code>prompts.py</code>	Prompt templates (stub today, will hold prompts the execution graph uses for LLM-side clarifica

13. Repository: assistant_core/execution/

Scaffolded today. When wired, this becomes the overnight pipeline that actually calls models and produces artifacts.

File	Responsibility
__init__.py	Marker.
graph.py	EXECUTION_STEPS list (the 18 named steps from load_ready_work_packets through finish_run)
file_utils.py	load_supported_file(path) for .txt, .md, .csv, .json, .log. profile_csv() returns row count, columns, p
evaluators.py	EvaluationResult parser. INVALID_EVALUATION fallback when an LLM evaluator returns malformed
output_writer.py	dated_output_dir(cfg, day) -> ~/LocalAI/output/YYYY-MM-DD/. write_status(dir, status) writes 00_9
prompts.py	Prompt templates for execution-time LLM calls (currently small).

14. Repository: assistant_core/llm/

File	Responsibility
__init__.py	Marker.
ollama_client.py	Low-level wrapper around Ollama's /api/generate and /api/tags. Used by code paths that bypass L
ollama_admin.py	Higher-level admin API used by the Streamlit Models tab: ollama_available, list_pulled_models, lis
litellm_client.py	chat_completion(messages, model_group, mode) wrapper that pre-checks the safety gate (assert_
model_policy.py	LOCAL_ESCALATION chains (general, coding, reasoning_check). local_model_chain(task_type)

15. Repository: assistant_core/memory/

File	Responsibility
<code>__init__.py</code>	Marker.
<code>obsidian.py</code>	<code>ensure_obsidian_vault()</code> creates the 10 standard subfolders. <code>write_daily_brief(content, day) -> ~/</code>
<code>memory_extractor.py</code>	<code>extract_memory_candidates(output_text, source_path)</code> scans markdown for lines containing 'dec
<code>memory_reviewer.py</code>	(stub today) Will gate which candidates make it into long-term memory based on your approval s
<code>vector_store.py</code>	ChromaMemoryStore wrapper. <code>connect()</code> creates a PersistentClient at the configured persist_dir

16. Repository: assistant_core/temporal_app/

File	Responsibility
__init__.py	Marker.
workflows.py	ClarificationWorkflow, OvernightExecutionWorkflow, ManualResumeWorkflow. Each is decorated with
activities.py	The seven activity functions, each decorated with @activity.defn. create_work_packet_activity is fully
worker.py	main_async() connects to Temporal at the configured address, creates a Worker registered for all wor
client.py	start_overnight_execution(work_packet_id) and start_clarification(title, description) submit workflows t

17. Repository: app/control_panel.py

The Streamlit UI. ~360 lines of declarative-ish Python; renders the 7 tabs described in section 8. Key structural elements:

Function	Responsibility
main()	Entry point. Sets page config, calls _render_header, builds tabs, dispatches to each _r
_render_header(cfg)	The two-row status strip + day-unlock banner + launchd banner + cloud warning.
_render_dashboard_tab()	Lists work packets from Postgres.
_render_create_packet_tab()	Form for drafting and scoring a packet without DB writes.
_render_clarification_tab()	File-upload preview for answers markdown.
_render_models_tab(mode, unlocked)	Currently loaded list, pulled models list, Load/Unload buttons, Quick test prompt.
_render_schedule_tab(cfg)	launchd status metrics, editable time form, regenerate-and-reload hint.
_render_execution_tab()	Shows the scaffold step list, disabled pause/resume buttons.
_render_artifacts_tab(cfg)	Walks ~/LocalAI/output/ and lists files.
current_mode()	Reads LOCALAI_MODE env var with DAY_MODE default.
_status_pill(label, ok)	Helper for the up/down metrics.

Important sys.path manipulation at the top: PROJECT_ROOT is computed and inserted at index 0 of sys.path so that from assistant_core... imports work even when Streamlit is launched from a different working directory.

18. Repository: scripts/ (every shell script)

Script	What it does
check_system.sh	Read-only health check: macOS, hardware, disk, Python, Homebrew, Docker, Ollama, po
install_core.sh	Creates .venv, pip installs dependencies, runs `assistant_core.cli ensure-folders` to mater
start_services.sh	docker compose up -d for the four containers. Verifies Docker daemon + compose plugin
stop_services.sh	docker compose stop. Does NOT touch host processes (Streamlit, Ollama, LiteLLM, work
status_services.sh	Inspection summary: what's running, what's listening.
start_litellm.sh	Copies config/litellm.yaml from .example if missing; runs litellm proxy bound to 127.0.0.1:4
start_temporal_worker.sh	Launches .venv/bin/python -m assistant_core.temporal_app.worker.
start_control_panel.sh	Exports PYTHONPATH and launches Streamlit on 127.0.0.1:8501.
create_work_packet.sh	Wrapper: .venv/bin/python -m assistant_core.cli create-work-packet "\$1" "\$2".
answer_questions.sh	Wrapper: .venv/bin/python -m assistant_core.cli answer-questions "\$1" "\$2".
run_clarification.sh	Wrapper for the run-clarification CLI command (rescore + regenerate questions for an exis
run_ready_overnight.sh	Refuses unless LOCALAI_MODE=NIGHT_MODE or LOCALAI_MANUAL_RESUME=true
pull_models.sh	Sequentially pulls the three default tags from Ollama; lists alternative tags at the end.
create_launchd_plist.sh	Reads night_mode_start from config/assistant.yaml; emits launchd/com.localai.orchestrat
day_unlock.sh	Writes ~/LocalAI/state/day_unlock.flag with timestamp + reason.
day_lock.sh	Removes the flag. Warns if LOCALAI_DAY_UNLOCK is also set in the shell.
run_tests.sh	shellcheck-equivalent (bash -n), py_compile sweep, pytest run, docker compose config va
build_pdfs.py	Generates the three PDF documents (System Guide, Onboarding, this Technical Referen

19. Repository: config/ (every YAML)

config/assistant.yaml

The main config. Sets paths, ports, thresholds, policies, and the editable day/night times. Read via `config.load_assistant_config()`. Editable through the Schedule tab (allow-listed to the time fields only).

Key	Default	Used by
local_ai_root	~/LocalAI	paths.ensure_local_folders
inbox_dir, output_dir, archive_dir, ...	subdirs of local_ai_root	various writers
obsidian_vault_dir	~/Obsidian/LocalAI-ChiefOfStaff	memory.obsidian
ollama_base_url	http://localhost:11434	ollama_admin, scheduler_status
litellm_base_url	http://localhost:4000	litellm_client
temporal_address	localhost:7233	temporal_app.worker, client
postgres_url	postgresql://localai:localai@localhost:5432/localai	postgres.connect
day_mode_start, night_mode_start, night_mode_end	07:00, 07:30, 07:00	scheduler_status.compute_window_status + c
minimum_readiness_for_overnight, _for_high_stakes	0.85, 0.95	readiness.readiness_status
cloud_fallback_enabled	false	safety.assert_cloud_allowed
daily_cloud_budget_usd	5.00	(future) cloud budget enforcement
temperature, top_p	0.2, 0.9	(future) model call defaults
max_chars_per_chunk	12000	execution.file_utils.chunk_text

config/models.yaml

The mapping of model group -> actual Ollama tag. Used by `ollama_admin.tag_to_group_mapping` (reverse-lookup) and implicitly by LiteLLM's config (which mirrors these values).

config/memory.yaml

Memory layer settings: whether Obsidian writes are enabled, the vector store choice (chroma), the vector store persist directory, and the per-policy knobs that the memory reviewer will use when wired up.

config/litellm.yaml (and .example)

LiteLLM's router config. `.example` is committed; `.yaml` is gitignored. `scripts/start_litellm.sh` copies `.example` to `.yaml` on first run if `.yaml` is missing.

20. Repository: db/migrations/

Postgres SQL applied automatically on first container boot via the docker-entrypoint-initdb.d bind mount. There is one file today:

File	Contents
001_init.sql	Creates the seven tables (work_packets, clarification_questions, execution_runs, model_calls, evaluation

Adding a new migration: drop a numbered file like 002_add_foo.sql. Postgres only runs init scripts on first boot (when the data directory is empty). For an existing DB you would either run the file manually via psql or destroy the postgres_data volume and let init re-run.

21. Repository: tests/ (every test file)

File	What it covers
conftest.py	Autouse fixture isolate_day_unlock that redirects safety.DEFAULT_STATE_DIR to a p
test_clarification_flow.py	End-to-end deterministic flow: vague request -> questions -> answers parsed -> resco
test_readiness.py	Readiness scoring edge cases including UNDERDEFINED, READY_FOR_OVERNIGH
test_safety.py	Every gate function. Parameterized over all four local-* groups for DAY_MODE block -
test_storage_serialization.py	Round-trip a WorkPacket through yaml serialize/deserialize.
test_file_utils.py	load_supported_file across .txt/.md/.log/.json, profile_csv shape and missing-value bel
test_control_panel_import.py	Streamlit app imports cleanly when the cwd is not the project root (regression for the s
test_scripts_static.py	Shell scripts contain expected gates (LOCALAI_WRITE_LAUNCHD check, docker cor
test_ollama_admin.py	tag-to-group resolution (incl. fail-closed fallback), gate enforcement, quick prompt trim
test_scheduler_status.py	Window math including overnight wrap, between-window gaps, launchd job detection,
test_config_writer.py	Allow-list rejection, HH:MM regex validation, atomic write leaves no .tmp leftovers, mis

22. Repository: docs/

File	Purpose
day_unlock.md	Full design rationale + manual for the day-unlock switch. Two channels, four
models_panel.md	Models tab design and code-level walkthrough. Tag-to-group resolution, fail-
auto_execution.md	Launchd job lifecycle: four states, arming via three deliberate terminal steps
Local_AI_Orchestrator_System_Guide.pdf	High-level architecture guide. 34 pages, 23 sections, TOC.
Local_AI_Orchestrator_Onboarding.pdf	Practical operator manual. 25 pages, 21 sections, 5 use-case recipes.
Local_AI_Orchestrator_Technical_Reference.pdf	This document. Every entity explained.

23. External state: ~/LocalAI/

The orchestrator's working directory outside the repo. Created by `install_core.sh -> ensure_local_folders()`. Owned by you; the orchestrator reads from `inbox/` and writes to `output/`, `archive/`, `logs/`, `state/`, `work_packets/`.

Path	What goes there
~/LocalAI/inbox/notes/	Your free-form notes (markdown). The system reads these but never modifies them.
~/LocalAI/inbox/transcripts/	Meeting / call transcripts (markdown). Same read-only contract.
~/LocalAI/inbox/docs/	Documents (markdown). Source material for briefs.
~/LocalAI/inbox/csv/	Tabular data (.csv). <code>file_utils.profile_csv</code> reads these.
~/LocalAI/inbox/cloud_review/	Files explicitly flagged for cloud review. Required path for the Claude gate to even con
~/LocalAI/output/YYYY-MM-DD/	Generated artifacts for that day's run. Regenerated each run; not durable.
~/LocalAI/archive/	Inputs that were processed are moved here after a run (will be, when execution graph
~/LocalAI/logs/	Runtime logs. Includes <code>launchd</code> stdout/stderr when armed.
~/LocalAI/state/	Operational state files. Currently: <code>day_unlock.flag</code> (if active). Future: chroma vector sto
~/LocalAI/work_packets/	(reserved) Per-packet files associated with longer-running work.

24. External state:

~/Obsidian/LocalAI-ChiefOfStaff/

Your long-term knowledge layer. Plain markdown, owned by you, indexed by Obsidian. The orchestrator writes selected outputs here as a parallel copy (briefs, packet summaries, memory candidates) so they survive even if you delete ~/LocalAI/output/.

Folder	What lands there
00_Inbox/Memory_Review/	Memory candidates awaiting your approval before joining long-term memory.
01_Daily_Briefs/	One markdown per day, written by the morning-brief pipeline.
02_Work_Packets/	Per-packet summary notes.
03_Projects/	Long-running project pages you maintain; orchestrator may append related links.
04_Meetings/	Meeting notes (yours + post-meeting outputs).
05_Decisions/	Logged decisions extracted from transcripts.
06_Stakeholders/	Per-person pages.
07_Playbooks/	Procedures and templates.
08_Prompts/	Curated prompts (yours or system-suggested).
09_System_Logs_Summaries/	Run summaries written after big overnight executions.
99_Archive/	Historical content.

25. State files: day_unlock.flag and others

File	Purpose	Owned
~/LocalAI/state/day_unlock.flag	Sentinel for the day-unlock switch. Existence opens the day-unlock switch. Tests are infor	day_unlock
~/LocalAI/state/chroma/	(future) Chroma vector store persist directory.	memory.vector_store
launchd/com.localai.orchestrator.nightly.plist	Generated plist (gitignored). Not loaded until you install launchd/launchd.plist.sh	launchd
~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist	Installed plist (only if you ran with LOCALAI_WRITE_LIBRARY_LAUNCH_AGENTS). Loaded	LaunchAgents

Inspection cheatsheet:

```
# Is day unlock on?
ls ~/LocalAI/state/day_unlock.flag 2>&1
cat ~/LocalAI/state/day_unlock.flag 2>&1

# Is the nightly job armed?
launchctl list | grep com.localai.orchestrator.nightly
ls ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist 2>&1
ls launchd/com.localai.orchestrator.nightly.plist
```

26. Environment variables in depth

Variable	Read by	Effect
LOCALAI_MODE	scripts/run_ready_overnight.sh, app/controller.py (default) / NIGHT_MODE / MANUAL_RESUME	DAY_MODE (default) / NIGHT_MODE / MANUAL_RESUME
LOCALAI_DAY_UNLOCK	safety.day_unlock_active	Truthy (1/true/yes/on, case-insensitive) opens the day lock.
LOCALAI_MANUAL_RESUME	scripts/run_ready_overnight.sh	Truthy lets run_ready_overnight.sh proceed outside of the day lock.
LOCALAI_WRITE_LAUNCHD	scripts/create_launchd_plist.sh	Truthy = install plist into ~/Library/LaunchAgents.
LOCALAI_POSTGRES_URL	(future) override the assistant.yaml postgres url	Convenience for testing.
LOCALAI_STATE_DIR	scripts/day_unlock.sh and day_lock.sh	Override the state directory (defaults to ~/LocalAI/state).
LOCALAI_PYTHON_BIN	scripts/install_core.sh	Force a specific Python interpreter (e.g. 3.13 instead of 3.11).
LOCALAI_PULL_LLAMA70B	scripts/pull_models.sh	Truthy = include the 70B model in the default pull.
LOCALAI_TEST_PYTHON	scripts/run_tests.sh	Override the Python used to run pytest.
ANTHROPIC_API_KEY	safety.assert_cloud_allowed (+ litellm.yaml)	Required (one of six gates) for cloud-claude-opus calls.
OPENAI_API_KEY	.env.example placeholder	Currently unused. Reserved if non-Anthropic cloud model.
OLLAMA_FLASH_ATTENTION, OLLAMA_SENTRY, OLLAMA_STATE_DIR	Ollama docs, Ollama State Dir (manually)	Performance knobs for Ollama; not set when running locally.

27. The complete port map

Port	Service	Process	Bound to
3000	Open WebUI	Docker container localai-open-webui	127.0.0.1
4000	LiteLLM	Host Python (litellm proxy)	127.0.0.1
5432	PostgreSQL	Docker container localai-postgres	127.0.0.1
7233	Temporal gRPC	Docker container localai-temporal	127.0.0.1
8233	Temporal UI	Docker container localai-temporal-ui	127.0.0.1
8501	Streamlit control panel	Host Python (streamlit run)	127.0.0.1
11434	Ollama	Host service (brew or foreground)	127.0.0.1

Every port binds to **127.0.0.1** only. Nothing exposed to the LAN. If you want LAN access you'd have to deliberately rebind — not recommended; the system was not designed to be accessed by anyone but you on your own machine.

Quick port check

```
for port in 3000 4000 5432 7233 8233 8501 11434; do
  if lsof -nP -iTCP:$port -sTCP:LISTEN >/dev/null 2>&1; then
    printf '%-5s in use\n' "$port"
  else
    printf '%-5s free\n' "$port"
  fi
done
```

28. Logging: where logs go, what they contain

Source	Log destination	Notes
Docker containers	docker logs <container>	Volatile; rotated by Docker.
Postgres (inside container)	docker logs localai-postgres	Includes init-script output on first boot.
Temporal server	docker logs localai-temporal	Verbose by default. Look for ERROR/WARN.
Ollama daemon	\$(brew --prefix)/var/log/ollama.log or stdout when backgrounded as brew service, brew handles it.	
LiteLLM	stdout of the litellm process (depends on how launched)	Redirect to a file if you want persistence.
Streamlit	stdout of the streamlit process	Redirect when launching with nohup.
Temporal worker	stdout	Redirect when launching with nohup.
launchd nightly job	~/LocalAI/logs/launchd.out.log and launchd.err.log	Hard-coded in the plist's StandardOutPath/StandardErrorPath
Ollama pull (when run via the user)	wherever you redirect	We use ~/LocalAI/logs/pull-<tag>.log convention
assistant_core's own logging (currently minimal)		Future: structured logs into ~/LocalAI/logs/.

29. Inspection cheatsheet

Postgres

```
# Open psql against the container
docker exec -it localai-postgres psql -U localai -d localai

# Last 10 work packets
SELECT id, title, status, readiness_score, created_at
FROM work_packets ORDER BY created_at DESC LIMIT 10;

# Unanswered questions for one packet
SELECT round_number, category, priority, question
FROM clarification_questions
WHERE work_packet_id = '<uuid>' AND answered = false
ORDER BY priority DESC;

# Recent overnight runs and their outcomes
SELECT id, work_packet_id, status, mode, started_at, completed_at
FROM execution_runs ORDER BY started_at DESC LIMIT 20;

# All model calls for a run (audit trail)
SELECT model_group, actual_model, success, prompt_chars, response_chars
FROM model_calls WHERE execution_run_id = '<uuid>';
```

Docker

```
docker ps                                # running containers
docker compose ps                        # same, from compose perspective
docker compose config                    # resolved compose yaml
docker volume ls                         # named volumes
docker volume inspect local-ai-orchestrator_postgres_data
docker exec -it localai-postgres bash    # shell inside Postgres container
```

Ollama

```
ollama list                             # tags on disk
ollama ps                               # tags in VRAM
ollama show qwen3:30b-a3b              # detailed metadata
ollama stop <tag>                       # force unload
ollama rm <tag>                         # delete from disk
curl http://localhost:11434/api/tags    # JSON list
curl http://localhost:11434/api/ps      # JSON in-VRAM
brew services list | grep ollama
tail -f "$(brew --prefix)/var/log/ollama.log"
```

Temporal

```
open http://localhost:8233              # UI in browser
docker logs --tail=50 localai-temporal
ps -ef | grep '[t]emporal_app.worker'   # is host worker running?
```

The orchestrator from Python

```
.venv/bin/python -m assistant_core.cli sample-readiness
.venv/bin/python -c "from assistant_core.safety import day_unlock_active; print(day_unlock_active())"
.venv/bin/python -c "from assistant_core.scheduler_status import compute_window_status; print(compute_window_status())"
.venv/bin/python -c "from assistant_core.llm.ollama_admin import list_loaded_models; print(list_loaded_models())"
```

End of Technical Reference. For high-level architecture, see System Guide; for operator recipes, see Onboarding.